

Week5

Task1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
int main() {
    int pipefd[2];
    pid_t pid;
    char buffer[100];
    int data[]={10, 20, 30, 40, 50};
    int n=5;
    if (pipe(pipefd)==-1) {
        perror("pipe");
        exit(1); }
    pid=fork();
    if (pid<0) {
        perror("fork");
        exit(1); }
    if (pid > 0) {
        close(pipefd[0]);
        printf("Producer (Parent):\n");
        for (int i=0;i<n;i++) {
            sprintf(buffer, "%d", data[i]);
            printf("Producing: %d\n", data[i]);
            write(pipefd[1],buffer,strlen(buffer)+1);
            usleep(100000);
        }
        close(pipefd[1]);
        wait(NULL);
    } else {
        close(pipefd[1]);
        printf("\nConsumer (Child):\n");
        while (read(pipefd[0],buffer,sizeof(buffer))>0) {
            printf("Consumed: %s\n",buffer);
        }
        close(pipefd[0]);
    }
    return 0;
}
```

Output:

```
Producer (Parent):
Producing: 10

Consumer (Child):
Consumed: 10
Producing: 20
Consumed: 20
Producing: 30
Consumed: 30
Producing: 40
Consumed: 40
Producing: 50
Consumed: 50
```

Task2

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

Source Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
    pid_t pid;
    pid=fork();
    if (pid<0) {
        printf("Fork failed\n");
        exit(1);
    }
    if (pid==0) {
        printf("Child process is executing ls -l using execl()\n");
        execl("/bin/ls", "ls", "-l", NULL);
        perror("execl failed");
        exit(1);
    } else {
        wait(NULL);
        printf("\nExecuting ls -l using system()\n");
        system("ls -l");
        printf("\nExecuting ls -l using fork() and pipe()\n");
        int pipefd[2];
        pipe(pipefd);
        pid_t pid2 = fork();
        if (pid2 == 0) {
            close(pipefd[0]);
            dup2(pipefd[1], STDOUT_FILENO);
            close(pipefd[1]);
            execl("/bin/ls", "ls", "-l", NULL);
            perror("execl failed");
            exit(1);
        } else {
            close(pipefd[1]);
            char buffer[1024];
            int n;
            while ((n = read(pipefd[0], buffer, sizeof(buffer) - 1)) > 0) {
                buffer[n] = '\0';
                printf("%s", buffer);
            }
        }
    }
}
```

```

    }
    close(pipefd[0]);
    wait(NULL);
}
return 0;
}

```

Output:

```

Child process is executing ls -l using execl()
total 768
drwxr-xr-x  6 root root  4096 Aug  7 14:42 OS
drwxr-xr-x 11 root root  4096 Aug  7 14:53 OSSP
drwxr-xr-x  3 root root  4096 Aug  7 14:41 OS_Lab
-rw-----  1 root root   731 Aug 14 05:47 b.c.save
-rw-----  1 root root  1693 Aug 14 05:58 bb.c.save

```

```

Executing ls -l using system()
total 768
drwxr-xr-x  6 root root  4096 Aug  7 14:42 OS
drwxr-xr-x 11 root root  4096 Aug  7 14:53 OSSP
drwxr-xr-x  3 root root  4096 Aug  7 14:41 OS_Lab
-rw-----  1 root root   731 Aug 14 05:47 b.c.save
-rw-----  1 root root  1693 Aug 14 05:58 bb.c.save

```

```

Executing ls -l using fork() and pipe()
total 768
drwxr-xr-x  6 root root  4096 Aug  7 14:42 OS
drwxr-xr-x 11 root root  4096 Aug  7 14:53 OSSP
drwxr-xr-x  3 root root  4096 Aug  7 14:41 OS_Lab
-rw-----  1 root root   731 Aug 14 05:47 b.c.save
-rw-----  1 root root  1693 Aug 14 05:58 bb.c.save

```